

University of Illinois Urbana-Champaign

Project Report Phase I

**CS513: Theory and Practice of Data Cleaning
Data Cleaning Project**

Po Lin, Charlene Khun, Madalyn Killian (Team 38)
pohungl2@illinois.edu, ckhun2@illinois.edu, killian7@illinois.edu
Dataset Chosen: NYPL What's on the Menu Dataset

Dataset Description: Schema Chart



Figure 1.1

Written Description

This dataset is a collection of information taken from restaurant menus, dating from the mid 1800s to the present day, with locations spanning across the world. The menu data is separated into four tables linked together by identification numbers, each shown in Figure 1.1 with its respective columns and headers. The following section addresses each table in order, starting with the leftmost, Menu.csv.

Menu.csv contains a physical description of the menu, along with the location it's from. Each menu has a primary key column 'id'. Notably, columns containing a 'name' and 'sponsor' value never contain an 'event', 'venue', or 'place' value, and vice versa. The 'location_type' column is completely empty and can be excluded when cleaning. The 'event' column contains frequent misspellings and typos, for example, "DINNER;", "DINNE", and "[DINNER]" all represent "DINNER", and this should be accounted for carefully during cleaning and sorting. The 'place' column has similar inconsistencies, compounded by the fact that each entry can represent a city, street, boat/at-sea location (common enough to note separately), country, exact location (restaurant name), or simply a "?".

MenuPage.csv contains identifiers linking the Menu.csv and MenuItem.csv datasets, along with the height and width of the menu. The 'menu_id' is a foreign key to the 'id' column in Menu.csv, however it is important to note that there are 'menu_id' values that are not present in the Menu.csv 'id' column, which presents some issues. In addition, there are multiple menu pages that have the same 'menu_id' value, implying there are sometimes multiple pages per each menu. This dataset does not contain any notable errors or typos. The 'full_height' and 'full_width' columns range from the 600s to 12,000s and 500s to 9,000s respectively, but no documentation specifies their unit of measurement. These values are consistent with pixel dimensions, though this could not be confirmed, and this is noted as a limitation of the dataset. The 'uuid' column seems to be a unique reference to the page however it is not a descriptive enough field for analysis purposes.

MenuItem.csv contains information regarding each specific menu item on the menu itself. The 'menu_page_id' field is a foreign key to the 'id' primary key column in the MenuPage.csv dataset. Multiple menu items frequently share the same 'menu_page_id', indicating that several items can appear on a single menu page. The 'dish_id' field is a foreign key to the 'id' primary key column in the Dish.csv dataset. Over 100 records in the dataset have an empty value for 'dish_id', suggesting that these items may not be dishes, or that the dataset is missing information. The 'xpos' and 'ypos' columns lack a specified unit of measurement, which is noted as a limitation of

the dataset. The most notable columns are 'price' and 'high_price'. The 'price' column may represent either the average cost of the item or the lowest recorded price across all instances of the item, though this is uncertain. The 'high_price' column is inferred to represent the highest recorded cost of the item across all menus on which it appears; when this value is null, it may indicate that the high price matches the 'price' column, though this too is uncertain and should be considered a limitation of the dataset.

Dish.csv contains data about all individual dishes across all menus, including their frequency, price data, year of introduction, and last year of appearance. As mentioned above, the 'dish_id' field in the MenuItem.csv dataset is a foreign key to the 'id' primary key column. Assuming the 'first_appeared' and 'last_appeared' columns represent years, some values are questionable, such as "1" and "2928," suggesting that these entries are either not actual years or were recorded incorrectly. The 'description' column is empty and should be noted as a limitation of the dataset. The 'lowest_price' and 'highest_price' columns also contain questionable values for some records; for example, several dishes list "0.0" for both the lowest and highest price, including interesting items such as Caviar, which is unlikely to have been priced at zero. Finally, the 'name' column contains a large number of typos, misspellings, and miscellaneous characters, such as "?? Fry?? Stew," ".... GUESTS....," ", Raw, on Half Shell," and ", Served with French, Mayonnaise or Russian DressingHearts of Lettuce." This column will require significant cleaning before it is viable for analysis.

Use Cases: Develop Three Use Cases

U0 / Zero Data Cleaning: A use case that requires little to no data cleaning is listing the raw transcribed dish items that appeared on a selected menu as well as the recorded menu items that have the greatest difference in the first/last appeared fields. The dataset already contains relationships between menus, menu pages, menu items, and dishes, so this use case can be answered by retrieving the existing records associated with a menu. Since this use case does not require price normalization, currency handling, or dish-name clustering, the original data is good enough as-is. Additionally, to evaluate longest standing menu items will only involve analysis on available first/last appeared field values which is already available in the dataset.

U1 / Main: Our main use case is to categorize menu price ranges (\$, \$\$, \$\$\$, \$\$\$\$) relative to the distribution of menu prices within comparable time periods. In order to achieve this analysis, we would need to restrict usable data to menus with usable dates and valid menu item prices. Data cleaning is necessary because:

- Menu item prices may be missing, malformed or inconsistent
- Menu dates and currencies may be incomplete or inconsistent
- Menus with too few priced items may produce misleading averages
- Dish names will need to be cleaned/normalized due to odd/vague attribute values
- Dish and menu items' prices would need to be inferred due to potentially missing prices between tables

We aim to get our data to a state where we are able to compare menus and their average dish prices grouped by similar time periods, as well as identify most/least expensive menus within those time periods.

U2 / Never Enough: Our targeted “never enough” use case outlines scenarios where we want to analyze menu affordability based on income and economic conditions of a particular restaurant/menu’s location at a given time period. Although this somewhat relates to our main use case (U1), affordability of menus and its associated items requires additional information that is not included in the dataset; we would need data points such as cost of living, inflation, tipping customs, tax rates, etc. to perform necessary analysis alongside the data we have available. Additionally, the current state of location information within the dataset is also too ambiguous for us to properly associate menus/restaurants to specific neighborhoods, cities or states. For example, a location value of “New York” does not properly distinguish between city, state, borough, or neighborhood, thus the identification of locality becomes a challenge even after data cleaning. Additionally, the distribution of menu item name values can be nonsensical. Even for our main use case, our assumptions are that some menu item/dish rows may actually refer to the same item on menus (which could explain poorly named and defined dishes and pricing data). Since we lack the proper unit of measurements for the menu dimensions (xpos, ypos, height, width), proper identification of items that could be referring to similar or identical locations on the menu page will prove to be challenging.

Data Quality Problems

The New York Public Library (NYPL) Menu dataset contains a large collection of historical restaurant menus collected over many decades. Although the dataset contains valuable historical information, our initial inspection revealed several obvious data quality problems that would prevent our main use case (U1) from producing accurate and reliable results. Since our objective is to categorize historical menu price ranges over comparable time periods, the data must first be cleaned and standardized before meaningful analysis can be performed.

Inconsistent Dish Names: One of the most noticeable data quality issues occurs in the Dish.csv dataset, where the same dish is represented using multiple text variations. Using OpenRefine's fingerprint keying function, we identified several clusters that clearly represent the same food item. For example, Figures 3.1 and 3.2 show several variations of Broiled Chicken.

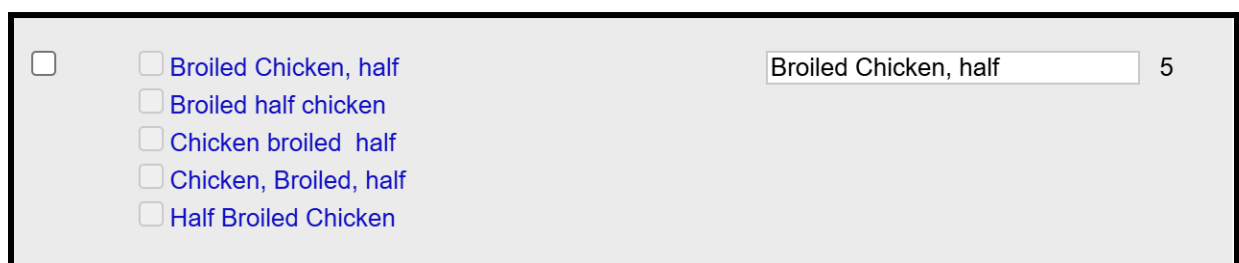


This figure shows an OpenRefine cluster interface. On the left, there is a master checkbox and a list of six variations of a dish name, each with its own checkbox. On the right, there is a text input field containing the selected variation and a count of the number of items in the cluster.

Selection	Dish Name Variation
☐	☐ Broiled Spring Chicken, half
	☐ Broiled spring chicken half
	☐ Chicken, Spring, broiled (half)
	☐ Half Broiled Spring Chicken
	☐ Spring chicken broiled, half
	☐ Spring chicken, broiled, half

Selected: Broiled Spring Chicken, half 6

Figure 3.1



This figure shows another OpenRefine cluster interface, similar to Figure 3.1 but for a different dish. It displays five variations of the dish name.

Selection	Dish Name Variation
☐	☐ Broiled Chicken, half
	☐ Broiled half chicken
	☐ Chicken broiled half
	☐ Chicken, Broiled, half
	☐ Half Broiled Chicken

Selected: Broiled Chicken, half 5

Figure 3.2

Without normalization, these dishes would be treated as separate items during analysis, reducing the consistency of dish grouping across menus. These values should therefore be merged into a standardized representation before analysis.

A similar inconsistency identified using the fingerprint keying function is that roast beef is represented in five different ways such as “Beef, Roast”, Roast beef with a comma and Roast beef as a period. (Shown in Figure 3.3).

These entries clearly describe the same dish but differ because of punctuation, capitalization, or word order. Standardizing these values into a single representation improves consistency throughout the dataset and reduces duplicate categories.



Figure 3.3

Numeric and Invalid Dish Names: Another issue discovered using the metaphone clustering algorithm is that several dish names consist entirely of numbers or isolated symbols instead of meaningful food names (shown in 3.4).

These values do not represent actual menu items and therefore provide no useful information for analysis. Since our project focuses on historical menu pricing rather than transcription artifacts, these records should either be removed or excluded from subsequent analysis.

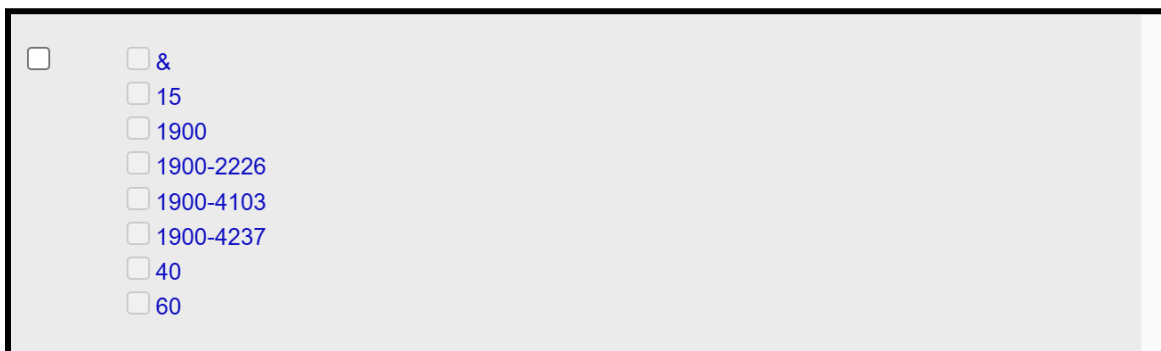


Figure 3.4

Misspelled Dish Names: In the below Figures 3.5 and 3.6, when using the cologne phonetic keying function, you can see that “Coffee” was spelled as “Cafee”, “Cofee”, and “Kafee” while “Tea ” was misspelled as “The” and “Thee.”

☐

☐ CAFE
☐ COFFEE.
☐ Cafee
☐ Cofee
☐ Coffee
☐ Guava
☐ Kaffee
☐ Kaffee,

8

[Browse this cluster](#)

Figure 3.5

☐

☐ Tea
☐ The
☐ Thee
☐ tea,

4

Figure 3.6

Although these values clearly refer to the same beverage, they are treated as different values within the dataset. Leaving these inconsistencies unchanged would create duplicate categories and reduce the quality of any grouping or aggregation performed later. These entries should therefore be merged under the correct standardized names.

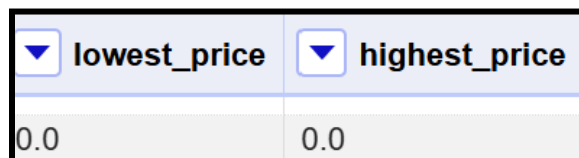
Broader Structural Data Quality Problems: Beyond these text inconsistencies, we also identified several broader structural data quality issues. Some of the prices in dish.csv are missing, indicated by a blank cell, as shown in Figure 3.7.

▼ lowest_price	▼ highest_price
0.25	20.0
0.0	0.0
0.0	0.95

Figure 3.7

Incomplete pricing prevents accurate calculation of menu averages and reduces the number of usable observations. Depending on the availability of additional information, these records may require imputation or removal before analysis.

We also notice that some of the prices are clearly incorrectly entered in (See Figure 3.8). This is because when you go to a restaurant, you would not expect them to sell free food. Typically, they should have a selling price greater than zero dollars.

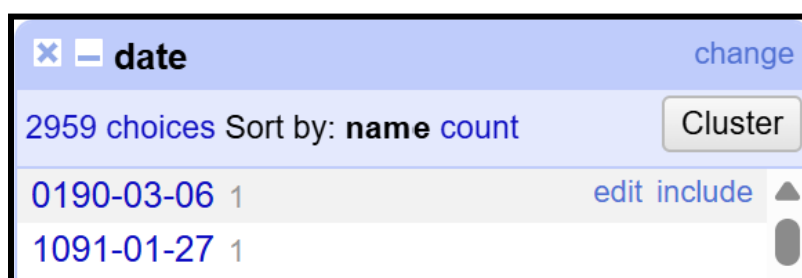


lowest_price	highest_price
0.0	0.0

Figure 3.8

Although a small number of promotional items could theoretically be free, historical restaurant menus generally list dishes with positive prices. A value of zero is therefore more likely to indicate missing or incorrectly entered information than an actual selling price.

Including these records would artificially reduce calculated menu averages and produce misleading menu classifications. Such values should therefore be corrected when possible or excluded from analysis. In Menu.csv, there are some interesting dates that do not make a lot of sense.



date		change
2959 choices Sort by: name count Cluster		
0190-03-06	1	edit include
1091-01-27	1	

Figure 3.9

Based on Figure 3.9, we would not expect a dish that was recorded in 1091 since many dishes were recorded mostly in the 1800s, 1900, and 2000s. It's just too old of a date from more than one thousand years ago.

Above the 1091 date also shown in Figure 3.9, we can see that 0190 is clearly a typo. These values are likely the result of transcription or data-entry errors. Because the

intended value cannot be verified, these records should be corrected only when supporting evidence exists, otherwise they should be excluded from analysis.

In Figure 3.10, we can see that at least 535 items did not have a date recorded at all and rendered as “blank”.

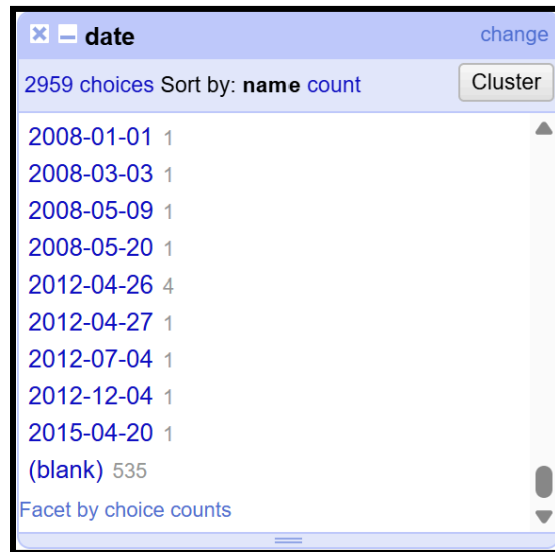


Figure 3.10

Another broad data inconsistency we noticed localized currencies are missing since the cells under currency and currency_symbol are empty. As shown in figure 3.11.

currency	currency_symbol

Figure 3.11

Since our main use case compares menu prices across historical time periods, undated menus cannot be reliably assigned to any chronological group. These records should therefore be excluded unless valid dates can be recovered from other sources.

Even when currency values are present, we notice that the dataset contains many different currencies (Figure 3.12) which can make the project more difficult for us since different currencies cannot be directly compared. To ensure meaningful price comparisons, our analysis will either standardize prices to a common currency or restrict the analysis to menus recorded in U.S. dollars.

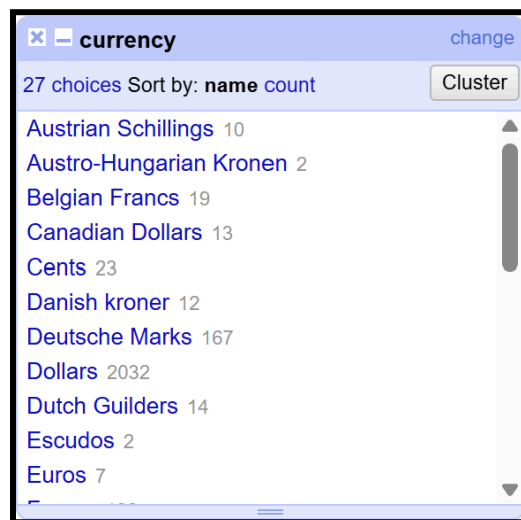


Figure 3.12

Going back to Dish.csv, when investigating the first_appeared column, we noticed that some of the dates did not numerically make sense.

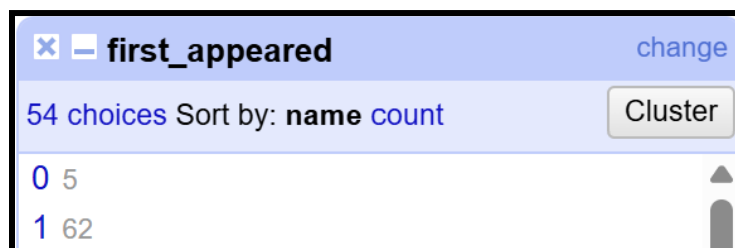


Figure 3.13

As seen in Figure 3.13, there are at least 5 values that were entered in as “0” and 62 values entered as “1”. We may want to exclude these values. A similar occurrence happened with the last_appeared column, where there are at least 5 values that were entered in as “0” and “2928”.

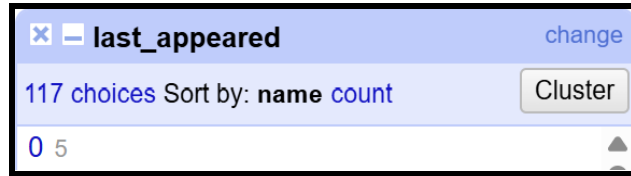


Figure 3.14

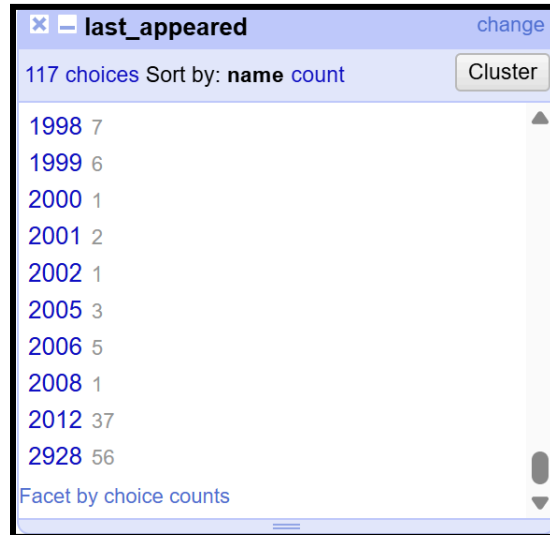


Figure 3.15

The year 2928 doesn't make sense at all because it's all the way in the future. We can infer that the author meant to enter in "1928" however, we most likely plan to not use those values to be safe.

And lastly, geographical identifiers can sometimes appear to be too ambiguous and broad. In figure 3.16, we have no idea just by looking where these locations are geographically located. For us as users, we would have to do more in depth research online to see where these places actually are.

☐ place
CAR ASTORIA
CAR SAVOY
CAR SAVOY
CAR APPIUS
CAR ASTORIA

Figure 3.16

The same thing goes with these different types of boats.

☐ place
S.S. "CITY OF PARA"
S.S. "DORIC"
S.S. "DORIC"
R.M.S. "EMPRESS OF CHINA"
R.M.S. "EMPRESS OF CHINA"

Figure 3.17

☐	☐ NEW YORK (69 rows)
	☐ [NEW YORK] (14 rows)
	☐ (NEW YORK?) (4 rows)
	☐ NEW YORK; (4 rows)
	☐ "NEW YORK" (3 rows)
	☐ (NEW YORK)
	☐ NEW YORK (?)
	☐ NEW YORK(?)
	☐ NEW YORK?

Figure 3.18

In figure 3.18, by doing some investigation using the cologne phonetic keying function, we can see that the author was ambiguous in this area. We cannot tell whether New York is a state or a city. These most likely will not be included as part of our data analysis.

How Data Cleaning is Necessary to Support U1: Our main use case aims to categorize historical restaurant menus into price ranges (\$, \$\$, \$\$\$, \$\$\$\$) by comparing average menu prices across similar historical time periods. Achieving this objective requires accurate menu dates, reliable menu prices, and standardized dish information. Without cleaning the dataset, the resulting classifications would be unreliable and potentially misleading.

Several of the identified data quality problems directly interfere with this analysis. Missing or invalid prices prevent accurate calculation of average menu costs, while zero-valued prices artificially reduce menu averages and distort restaurant price classifications. Similarly, menus containing missing or impossible dates cannot be assigned to the correct historical period, making chronological comparisons inaccurate. Multiple currency systems also prevent direct comparison of menu prices because identical numerical values may represent entirely different purchasing power.

Text inconsistencies further reduce data quality. Different spellings, punctuation, capitalization, and word order cause identical dishes to appear as separate records. Standardizing these values ensures that equivalent dishes are consistently represented throughout the dataset and prevents unnecessary duplication during analysis.

To prepare the dataset for analysis, several cleaning operations will be required. These include removing leading and trailing whitespace, standardizing capitalization and punctuation, merging equivalent dish names using clustering algorithms such as Fingerprint and Cologne Phonetic, filtering records with impossible dates or invalid price values, handling missing values where appropriate, and restricting analysis to records with complete pricing and chronological information. Currency values must also be standardized or filtered so that meaningful price comparisons can be made.

These cleaning steps are both necessary and sufficient to support our main use case. Once prices, dates, currencies, and dish names have been standardized, menus can be grouped into comparable historical periods, average menu prices can be calculated accurately, and restaurants can be classified into meaningful price ranges. No additional external information is required to perform U1 after these data quality issues have been addressed, allowing the cleaned dataset to support reliable historical menu price analysis.

Important Notes: Although Part A identifies several data quality issues involving menu item names (such as inconsistent naming, misspellings, punctuation differences, and duplicate representations), our inspection revealed that these textual attributes are substantially more difficult to clean reliably than the numerical attributes required for our main use case (U1). Fully standardizing historical dish names would require extensive manual interpretation because many names are ambiguous, incomplete, or represented in numerous different ways. As a result, we have decided to design U1 to focus on historical menu price analysis, where the required cleaning tasks such as correcting invalid or missing prices, removing impossible dates, handling missing values, and standardizing currencies are more objective and can be completed consistently. This ensures that data cleaning is both necessary and sufficient to support U1, while also providing a clear justification for why our project emphasizes numerical data rather than attempting to analyze unreliable textual menu item names.

Initial Plan for Phase II

S1 - Reviewing Case Description: This step involves reviewing the use case description and dataset description to confirm that the selected data is sufficient and available for collection. The data identified for this use case includes the following:

- Menu.csv: id, date, currency, currency_symbol, status (only looking at 'Complete')
- MenuPage.csv: id, menu_id
- MenuItem.csv: id, menu_page_id, price, high_price, dish_id
- Dish.csv: id, name, lowest_price, highest_price

The chosen raw datasets will be collected into a local project repository, with original files kept unchanged. This repository will expand over time to house scripts, outputs, and additional documentation as the project progresses.

S2 - Profiling and Identifying DQ Problems: This step involves profiling the data and identifying data quality problems relevant to the use case. Profiling will be conducted using a combination of tools: OpenRefine for initial data inspection, Python for loading the data and applying regex-based parsing and type conversions, SQL for efficient data retrieval and manipulation, and ChatGPT for supplementary support. OpenRefine is intended primarily for profiling and review rather than as the main cleaning tool, given the size of the dataset and the memory demands of processing larger files. All columns relevant to the U1 use case will be profiled to identify issues such as missing, blank, or unusual values, invalid-looking dates, currency inconsistencies, zero or suspicious prices, malformed numeric fields, and other outliers. This profiling step establishes what the Python/Pandas cleaning logic will need to handle.

S3 - Performing Data Cleaning: The main data cleaning workflow will be performed through an iterative pipeline using a combination of local file management, OpenRefine, Python/Pandas, MySQL, and AI-assisted code generation where appropriate and permitted by course guidelines. The intended cleaning workflow is outlined below.

First, observed problems will be converted into explicit cleaning requirements, establishing expectations for the target data type of each relevant column. Examples of these data quality requirements include:

- IDs should be preserved and converted into consistent ID formats where possible.
- String fields should be trimmed and standardized where appropriate.

- Date fields should be parsed into usable years or time periods.
- Price fields should be converted into valid numeric values.

Additional requirements may be identified as the cleaning process progresses.

Python/Pandas will then be used to clean the raw CSVs and generate cleaned CSVs that can later be loaded into SQL. The purpose of this step is to modify the raw files into the correct shape and expected data types, not to build the final joined analysis model. Cleaning logic will be organized by data type and column role: the data type of a field determines which cleaning tasks apply, while its role (e.g., identifier, date, price) determines the target state it should be cleaned toward. The data and column types anticipated for cleaning are outlined below.

Field Type	Strategy	Examples
Identifier / Key	• preserve original ID values • trim whitespace if needed • convert into consistent numeric or string ID format • flag missing or malformed required IDs • do not invent missing IDs • keep original row numbers for provenance	• Menu.id • MenuPage.id • MenuPage.menu_id • MenuItem.id • MenuItem.menu_page_id • MenuItem.dish_id • Dish.id
String / Categorical	• trim leading/trailing whitespace • standardize obvious capitalization where appropriate • normalize blank/null-like values • preserve original values when useful for provenance • create cleaned versions of important categorical fields • establish acceptable values (enums)	• Menu.status • Dish.name
Date / Time-Period	• trim whitespace • parse usable years from date values • create 'cleaned_year' field • flag missing, malformed, or impossible dates • mark rows with unusable dates for exclusion from U1	• Menu.date
Currency	• trim whitespace	• Menu.currency

	• standardize obvious currency labels and symbols • compare 'currency' and 'currency_symbol' when both exist • create 'currency_clean' or 'currency_scope' field • flag missing, ambiguous, or non-comparable currencies • mark only dollar-denominated/comparable menus as usable for U1 (pending further review)	• Menu.currency_symbol
Numeric Price	• preserve original price values • trim whitespace • convert values into numeric columns where possible • flag blanks and malformed values • flag zero or negative values • flag suspicious/extreme values for review • create cleaned numeric price columns	• MenuItem.price • MenuItem.high_price • Dish.lowest_price • Dish.highest_price

A specific fallback logic was defined following preliminary profiling of the price properties in the MenuItem and Dish datasets, due to the significant number of missing values observed during profiling. The following logic determines menu item and dish pricing:

1. Use MenuItem.price if present and valid.
2. Else, use MenuItem.high_price if present and valid.
3. Else, use the average of Dish.lowest_price and Dish.highest_price if both are present and valid.
4. Else, use Dish.lowest_price if valid.
5. Else, use Dish.highest_price if valid.
6. Else, mark the item as having no valid price and exclude it from price-based analysis.

To preserve provenance, the fields 'clean_price_candidate' and 'price_source' will be added to the MenuItem cleaned CSV, recording each item's final determined price along with its source. Examples of 'price_source' values include 'menu_item_price', 'menu_item_high_price', 'dish_lowest_highest_average', 'dish_lowest_price', 'dish_highest_price', and 'no_valid_price'.

During the Python/Pandas cleaning step, fields will be created to indicate whether a given row should be included in the final output, rather than dropping rows outright. The following fields will act as flags to support easier querying, as well as provenance tracking and D to D' analysis:

- cleaning_status
- warning_reason
- exclusion_reason
- source_file
- source_row_num

Example reasons for flagging include, but are not limited to:

- missing_required_id
- malformed_id
- missing_date
- invalid_date
- missing_currency
- ambiguous_currency
- missing_price
- non_positive_price
- malformed_price
- no_valid_fallback_price

After the Pandas cleaning step, the following cleaned versions of the original raw CSVs will have been created, along with additional logging to support the provenance analysis in S4:

- Menu_cleaned.csv
- MenuPage_cleaned.csv
- MenuItem_cleaned.csv
- Dish_cleaned.csv
- Cleaning_log.csv

SQL staging tables will then be defined to house these cleaned files. If the Python/Pandas cleaning logic changes in later iterations, the staging schema may also need to be revised. Staging table names used to store cleaned raw data include 'stg_menu', 'stg_menu_page', 'stg_menu_item', and 'stg_dish'.

At this point, the data will already have been cleaned and type-normalized by Python/Pandas. The cleaned data will then be loaded into SQL, which serves as the validation layer. The preceding cleaning steps are intended to ensure the data conforms to the expected data types. Once the cleaned data is in staging tables, SQL scripts will be run to check whether the staged data satisfies the domain validation constraints required for U1. These constraints include:

- ICs (integrity/domain constraints): e.g., "Menu.cleaned_year should be valid if the menu is used for U1."
- INDs (inclusion dependency / relationship checks): e.g., "MenuPage.menu_id should reference Menu.id."
- FDs (functional dependency checks): e.g., "Dish.id → name, cleaned_lowest_price, cleaned_highest_price"

After running SQL validation checks, any rows that violate ICs, INDs, FDs, or domain rules will be exported into separate CSV files by violation type, for example:

- validation_ic_results.csv
- validation_ind_results.csv
- validation_fd_results.csv

These violation files will then be examined individually, and the cleaning and validation process will be repeated until the staged data is acceptable for the U1 use case. At each iteration, output rows will be retained as a form of provenance and documentation for later stages of the project.

Once staging validation is acceptable, the final operational dataset for U1 will be defined. The targeted final tables are as follows:

Table Name / Possible Fields:

- final_menu: menu_id, cleaned_year, currency_clean, status_clean
- final_item: menu_item_id, menu_id, menu_page_id, dish_id, clean_item_price, price_source, original_price, original_high_price, dish_lowest_price, dish_highest_price

Once cleaning is complete, the final D' dataset, validation reports, excluded records, and provenance logs will be exported. Final outputs will include:

- final_menu.csv
- final_item.csv
- cleaning_log.csv

- excluded_records.csv
- validation_report.csv

S4 - Data Quality Checks: As much of the groundwork for data quality checks was established in S3, this phase focuses primarily on analyzing the resulting output files. After obtaining the cleaned source CSVs detailed in S3, it will be verified that they have preserved the original structure of the NYPL dataset while incorporating cleaned and type-normalized fields. Key questions to address include:

- Did each cleaned CSV load successfully into SQL staging tables?
- Do the cleaned columns match the expected data types?
- Were original IDs and source row identifiers preserved?
- Which fields were changed, standardized, or flagged?
- Which rows were marked as unusable?

After obtaining the SQL validation outputs defined in S3, these files will be reviewed to analyze the rows that failed SQL validation checks (IC/IND/FD). Key questions to address about these files include:

- Which integrity constraints passed?
- Which integrity constraints failed?
- Which inclusion dependencies failed?
- Which functional dependency checks showed violations?
- How many rows violated each rule?
- Are violations severe enough to require another cleaning iteration?
- Can remaining violations be excluded safely from U1?

After obtaining the violation CSVs defined in S3, it will be verified that they capture all data related to inconsistencies found in the raw dataset. Key questions to address about these files include:

- Are the violations caused by dirty raw values or by the cleaning logic itself?
- Do the violation rows reveal a new pattern that was not previously accounted for?
- Can these records be repaired with a better cleaning rule?
- Should these records be excluded from the final operational dataset?
- Do we need to repeat the S3 cleaning steps?

Based on the cleaning logs produced, the applied rules will be reviewed and summarized. Key questions to address about these files include:

- What cleaning rules were applied?
- Which columns did each rule affect?
- How many rows were changed, flagged, or excluded by each rule?
- Were new rules added after reviewing validation failures?
- How did the pricing fallback logic perform, and how many records were recovered as a result?
- Can the transformation from D to D' be explained using this log?

The before/after validation summary will address the following questions:

- How many missing dates existed before cleaning?
- How many missing or invalid prices existed before cleaning?
- How many ambiguous currency values existed before cleaning?
- How many broken references existed before cleaning?
- How many records were repaired, standardized, or excluded?

S5 - Documenting and Quantifying Change: This section documents the transformation from the original dataset (D) to the cleaned dataset (D'), using a 10,000-row working sample from Menu.csv and a 10,000-row working sample from Dish.csv. The measurements below distinguish between problems detected in the original values, derived status or eligibility cells created during cleaning, and rows retained or excluded from the operational dataset for U1. Since U1 compares historical menu prices across comparable time periods, the analysis emphasizes dates, currency, prices, and year ranges.

For Menu.csv, four derived columns were added during cleaning: new_date, currency_new, status, and new_test. Original source values were preserved throughout this process; uncertain values were classified, flagged, or excluded rather than silently replaced. As a result, the counts below reflect both the data-quality violations identified in the original data and the cleaning decisions applied to address them.

The following measurements are based on an initial 10,000-row sample cleaned using OpenRefine. See page 22.

Measurement in Menu.csv	Initial Data Cleaning Before	Initial Data Cleaning After	Cleaning Interpretation
Rows evaluated	10,000	10,000	All rows retained for assessment
Invalid Dates	540	0	535 missing dates were flagged; 5 additional invalid date values were identified and handled by the date rule
Missing and Non usable currency	7968	0	7,434 missing and 534 noncomparable values were assigned an explicit currency status

Measurement in Menu.csv (continued)	Initial Data Cleaning Before	Initial Data Cleaning After	Cleaning Interpretation
Rows Under Review	85	0	All 85 rows were explicitly marked under_review
Usable Candidates	Not determined before cleaning	1795	Rows passed date, currency, and review eligibility rules

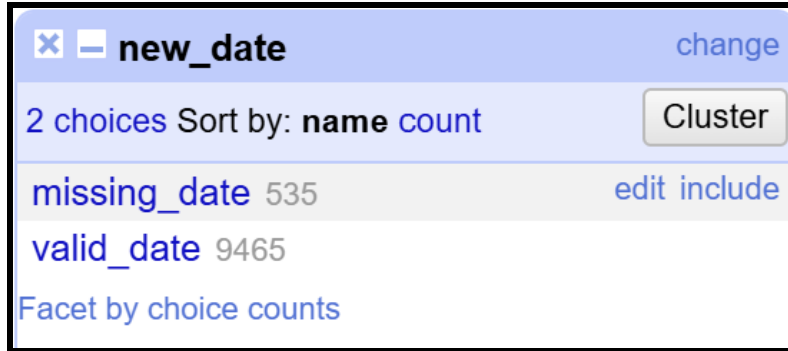


Figure S5.1: Missing and invalid date values identified using blank and text facets.



Figure S5.2: Missing Currency and Non Comparable Currency

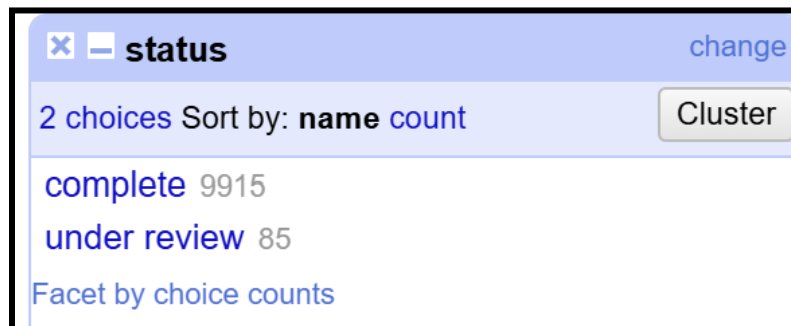


Figure S5.3: Number of Completed and under review status

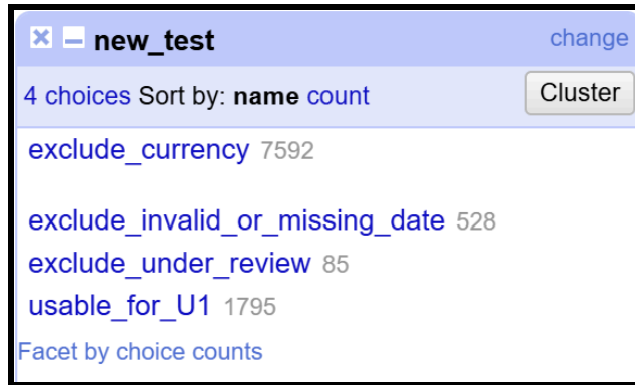


Figure S5.4: Number of Usable items for Menu.csv

For Dish.csv, three derived columns were added: first_appeared_status, last_appeared_status, and usable_candidate. As with Menu.csv, original values were preserved, and uncertain values were classified, flagged, or excluded rather than silently replaced.

Measurement in Dish.csv	Initial Data Cleaning Before	Initial Data Cleaning After	Cleaning Interpretation
Blank Lowest Price	329	329	The blanks were preserved and considered by the usable-price rule
Blank Highest Price	329	329	The blanks were preserved and considered by the usable-price rule
First Appeared Year doesn't make sense	67	0	Each value was labeled valid or invalid_too_early
Last Appeared Year doesn't make sense	61	0	56 future-year and 5 too_early values were explicitly classified

Measurement in Dish.csv (continued)	Initial Data Cleaning Before	Initial Data Cleaning After	Cleaning Interpretation
Usable Candidates	Not determined before cleaning	6489	Rows passed price and year eligibility rules



Figure S5.5: Blank Missing Lowest Prices and Highest Prices



Figure S5.6: Invalid Years in First Appeared column

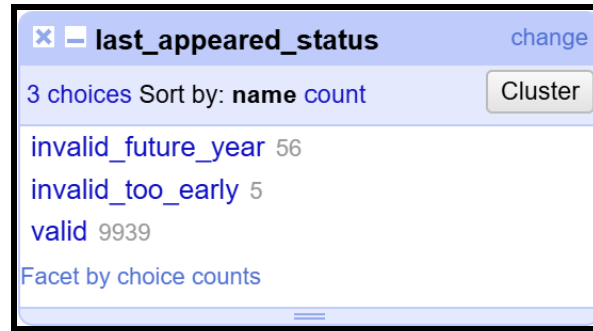


Figure S5.7: Invalid Years in Last Appeared column



Figure S5.8: Usable Items in Dish.csv

When integrity constraints for both Menu.csv and Dish.csv were checked specifically for non-unique IDs, no violations were found and every record had a unique ID. The following table is an integrity constraint summary.

Integrity Constraint	Before Cleaning	After Cleaning
Date domain: Menu.date must be usable for a historical period	540 violations detected	0 unclassified; all were flagged or handled
Currency domain: currency must be comparable for U1	7,968 violations detected	0 unclassified; all received a currency status
Price rule: Dish must have at least one usable positive price	3,066 final violations	3,066 excluded from U1 rather than imputed

Integrity Constraint	Before Cleaning	After Cleaning
Year domain: first_appeared and last_appeared must fall in the accepted range	67 first-year and 61 last-year violation	0 unclassified; invalid rows were flagged and excluded by the final rule

Overall, the cleaning process successfully documented and quantified the transformation of the sampled data into records suitable for U1. In Menu.csv, 540 date violations, 7,968 missing or non-comparable currency values, and 85 records requiring review were identified and assigned explicit statuses, resulting in 1,795 records that satisfied the date, currency, and review requirements for historical price comparison. In Dish.csv, 67 invalid first-appeared years, 61 invalid last-appeared years, and 3,066 records without a usable positive price were identified, flagged, and excluded when necessary, leaving 6,489 usable dish candidates. Original source values were preserved rather than silently replaced, and no duplicate-ID violations were found in either sampled table. These results demonstrate that the cleaning process reduced ambiguity, documented the remaining integrity-constraint violations, and produced a more reliable operational dataset for comparing historical menu prices across valid time periods.

Divided Phase I Tasks

Po Lin (pohungl2@illinois.edu): Develop 3 Use Cases; S3, S4 of Initial Plan for Phase II; Team Leader; edited and provided feedback

Charlene Khun (ckhun2@illinois.edu): List obvious data quality problems; S5 of Initial Plan for Phase II; edited provided and provided feedback; hosted meetings

Madalyn Killian (killian7@illinois.edu): Describe your chosen dataset; S1 and S2 of Initial Plan for Phase II); edited and provided feedback

Tentative Phase II Tasks

Po Lin (pohungl2@illinois.edu): Python Scripts, AI Supplementary Resources (Chat GPT, Gemini, Claude, etc.), Section 2

Charlene Khun (ckhun2@illinois.edu): Open Refine, SQL Scripts, Section 1, Section 4

Madalyn Killian (killian7@illinois.edu): SQL Scripts, YesWorkflow, Section 3